BearShare Security Design Document

Computer Security CS419

Paige Brathwaite & Smila Gala-Alfonso

April 6th, 2026

1. Introduction

2. Architecture Overview

3. Threat Model

4. Security Controls

5. Data Protection

1. Introduction

BearShare was implemented as a Flask-based web application using file-based JSON storage instead of a database, in accordance with the project requirements. This design choice kept the system lightweight and manageable while still allowing the application to support persistent user accounts, session tracking, document metadata, sharing permissions, audit history, password changes, and administrator-managed global documents. Although the implementation is simplified for a course project, the final system reflects a broader set of secure file-sharing concerns, including role-aware access control, encrypted storage, versioned uploads, HTTPS testing with a self-signed certificate, and verification of browser-side protections such as security headers and secure session cookies.

The purpose of this project is not only to create a working web application, but also to demonstrate how secure design principles can be integrated into a realistic file-sharing workflow. This means that user credentials, uploaded files, session data, access permissions, and audit records are all treated as important assets that are protected against misuse, unauthorized access, and common web attacks.

BearShare was implemented as a Flask-based web application using file-based JSON storage instead of a database, in accordance with the project requirements. This design choice kept the system lightweight and manageable while still allowing the application to support persistent user accounts, session tracking, document metadata, sharing permissions, and audit history. Although the implementation is simplified for a course

project, the overall architecture reflects many of the same security concerns that would appear in a real-world secure file-sharing system.

2. Architecture Overview

BearShare follows a client-server architecture in which users interact with the system through a browser-based interface and submit requests to a Flask backend. The frontend is responsible for collecting user input and presenting files, forms, and account-related actions, while the backend enforces all important security decisions. This separation is important because BearShare does not rely on the user interface alone to protect data. Even if a user manually changes a URL or attempts to bypass visible restrictions in the browser, the server still performs its own authorization and validation checks before allowing any sensitive action.

At the center of the system is the main Flask application in app.py, which handles routing, request processing, and coordination between the major support modules. BearShare uses dedicated helper modules for authentication, session management, validation, encryption, document operations, and logging. These modules help separate responsibilities so that the application remains easier to understand and maintain. For example, login and registration logic are handled in the authentication module, while document sharing, permission logic, audit records, and versioning are handled in the document service module.

Persistent state in BearShare is stored using JSON files rather than a relational database. This includes user accounts, active sessions, document metadata, sharing

relationships, audit history, and failed login tracking. Uploaded document contents themselves are not stored in plaintext JSON. Instead, the actual file bytes are encrypted and written to a dedicated storage directory on disk. This allows BearShare to separate sensitive file contents from metadata while still preserving the ability to control access and retrieve the latest version of a document when needed.

The architecture also includes a logging layer that records both security-relevant events and general access activity. Security logs capture events such as failed login attempts, denied actions, and administrative role changes, while access logs capture user actions such as successful uploads, downloads, and document sharing. Together, these logs support accountability and make it easier to review how the system was used during testing.

A simplified architecture diagram for BearShare would show the browser at the top, followed by the Flask application layer, then the supporting service modules, and finally the persistent storage and encrypted file layer beneath them. In practice, the flow is: user request, Flask route, validation and permission checks, service-layer logic, storage or retrieval from JSON and encrypted file storage, and then a response back to the user.

Component Descriptions

The main Flask application acts as the entry point for BearShare and is responsible for coordinating the overall workflow of the system. It handles registration, login, logout, dashboard rendering, document upload and download, file viewing, document sharing,

administrative role management, and security-related behaviors such as HTTPS enforcement and security headers.

The authentication module is responsible for validating user registration input, hashing passwords with bcrypt, checking credentials during login, tracking failed login attempts, enforcing account lockout, and handling IP-based login rate limiting. This module protects one of the most sensitive parts of the system: access to user accounts.

The session manager creates and validates session tokens, tracks session timestamps, and expires sessions after inactivity. This ensures that BearShare does not simply trust a login once it has happened, but continues to check whether a session remains valid over time.

The document service module manages the core document logic of the platform. It creates document metadata, assigns ownership, handles document sharing, enforces document-specific roles such as owner, editor, and viewer, updates document versions, performs soft deletion, and records audit events for important actions. Because BearShare uses both system-wide roles and document-specific roles, this module is essential to enforcing access correctly.

The encryption component is responsible for protecting uploaded documents at rest. Instead of writing plaintext file contents to disk, BearShare encrypts file bytes before storing them and decrypts them only when an authorized user requests access. This provides an important layer of protection in case stored files are exposed at the filesystem level.

The validation module enforces safety checks on user-controlled data, including username format, email validity, password strength, allowed file types, and safe filename

handling. In the final implementation, upload protection goes beyond simple extension filtering. BearShare checks allowed extensions, validates the reported MIME type, and compares the uploaded file's content signature against the declared file type before accepting the upload. These checks are applied both to initial uploads and to versioned document updates, helping reduce the risk of malformed, mislabeled, or suspicious files entering the system.

Technology Stack Justification

BearShare uses Python and Flask because Flask provides a lightweight and flexible framework for building a security-focused web application without unnecessary complexity. Since the project required custom implementations of authentication, access control, session handling, encryption, and logging, Flask was a good fit because it allowed the system to be built in a modular way while still remaining small enough to understand fully.

File-based JSON storage was chosen because the project specification explicitly allows file-based persistence and does not require a database. This choice made it easier to inspect application state directly during development and testing, especially when verifying user roles, session data, sharing relationships, and audit records. While this storage model would not scale well to production, it is appropriate for a course project and supports the required functionality clearly.

The bcrypt library was selected for password hashing because it is a widely accepted and secure method for storing passwords. The cryptography library was selected for file encryption because it offers a straightforward and reliable implementation of symmetric

encryption for protecting stored document contents. Jinja-based HTML templates were used for the frontend because they integrate naturally with Flask and support automatic escaping, which helps reduce XSS risk during rendering.

Data Flow Diagrams

The following diagrams show the two most security-sensitive workflows in BearShare: authentication/session establishment and upload/storage. In both cases, the server performs validation and authorization before any sensitive state change occurs.

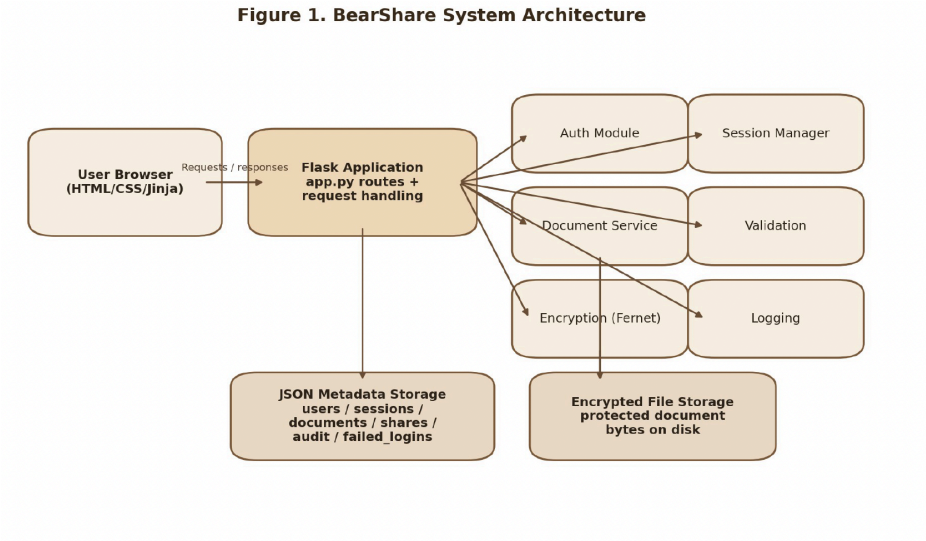

Figure 1. High-level architecture of the BearShare application.

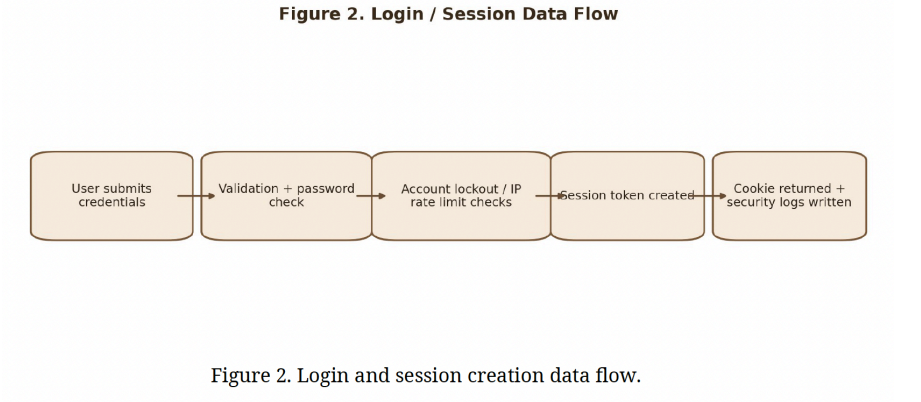

Figure 2. Login and session creation data flow.

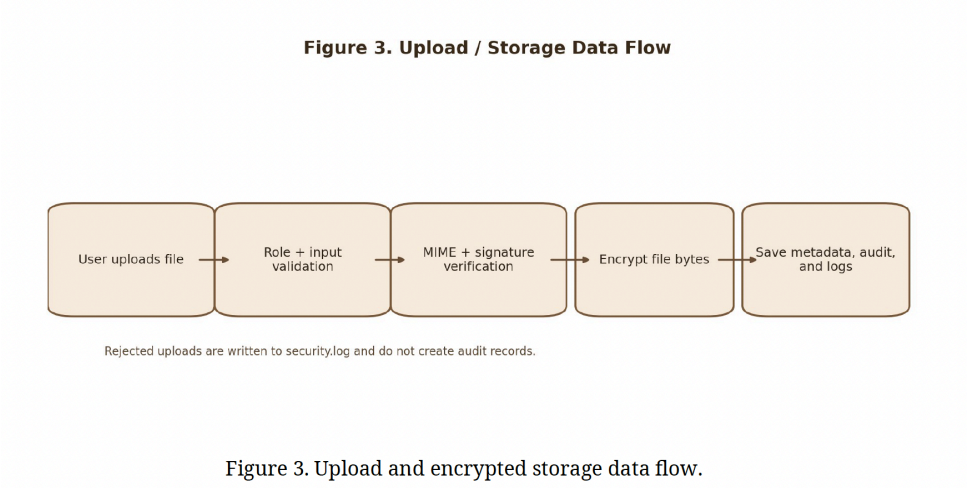

Figure 3. Upload and encrypted storage data flow.

3. Threat Model

The threat model for BearShare begins with identifying the assets that the system must protect. These include user credentials, password hashes, session tokens, confidential uploaded documents, document-sharing permissions, role assignments, audit records, log files, and encryption keys. Each of these assets has security value because unauthorized access, modification, or disclosure could undermine the confidentiality, integrity, or availability of the platform.

Several threats are relevant to BearShare because of the way users interact with the system. Since the platform includes user accounts and protected files, brute-force login attempts and unauthorized document access are major concerns. The application's use of different roles, both system-level and document-level, makes privilege escalation also a

meaningful threat. A user may attempt to gain access to documents they do not own, access administrative functionality, or continue performing write actions after being downgraded to a more limited role. The platform must also consider common web application threats such as cross-site scripting, path traversal, malicious file uploads, session misuse, and insecure transport.

A vulnerability assessment of BearShare focuses on the places where user-controlled input meets security-sensitive operations. Login routes are vulnerable to brute-force attacks if lockout and rate limiting are not implemented properly. File upload routes are sensitive because they process user-supplied files and filenames. Sharing and document routes are sensitive because improper authorization checks could allow one user to access or modify another user's data. Administrative routes are especially sensitive because they can change account roles and affect access to the rest of the platform. The system must therefore assume that users may intentionally manipulate URLs, form inputs, uploaded files, and session state in order to test or exploit weaknesses.

Several realistic attack scenarios illustrate these risks: 1.) An attack scenario involves a user attempting to access another person's file by manually changing the document identifier in a download or view route; 2.) A second involves repeated failed login attempts intended to guess a valid password; 3.) A third involves a guest account trying to obtain or retain editor-level access to a document even though that role should be restricted; 4.) A fourth scenario involves an attacker trying to inject script-like input into forms in the hope that it will later be rendered unsafely; 5.) A final example is a path traversal attempt in which suspicious file-related input is used to try to reference files outside the intended storage directories.

In terms of risk prioritization, the highest-priority threats in BearShare are unauthorized document access, privilege escalation, authentication abuse, and exposure of stored documents. These risks directly affect the main purpose of the platform, which is secure file sharing. Session misuse and insecure transport are also high priority because they can undermine otherwise-correct authentication and authorization. Input-based attacks such as XSS and path traversal are also important, though their actual impact depends on whether the application improperly trusts or renders user input.
Lower-priority risks are generally those that affect polish or convenience more than core confidentiality or access control, but they still need to be addressed where possible.

4. Security Controls

BearShare uses multiple layers of security controls so that no single mechanism is responsible for protecting the entire application. Instead of relying on only one defense, the system combines authentication controls, role-based access control, input validation, encrypted storage, session protections, security headers, and logging. This layered approach is important because secure systems must assume that users may attempt to misuse available functionality and that mistakes at one layer should still be caught by another. In addition to the manual testing described throughout this section, bearshare includes a small automated unit test suite for several core security behaviors. These tests cover registration and login behavior, account lockout, password change, upload validation, access control enforcement, and session invalidation. This helped verify that important

protections continued to behave correctly in a repeatable way as the application was updated.

Authentication

BearShare protects user accounts through registration validation, bcrypt password hashing, account lockout, and IP-based rate limiting. Registration checks username, email, and password strength before account creation, while login tracks failed attempts per account and per IP to reduce brute-force abuse. Authentication was tested through weak and duplicate registrations, repeated failed logins, and multi-account IP abuse attempts. Limitations include the lack of password reset or MFA, but these risks are partially mitigated by strong password rules, lockout enforcement, rate limiting, and logging.

Access Control

BearShare enforces access control at both the system level and the document level through admin, user, and guest roles as well as owner, editor, and viewer permissions. Authorization is enforced on the backend rather than only in the interface, which prevents users from bypassing restrictions through direct URL manipulation. Access control was tested by attempting unauthorized admin access, changing document IDs manually, and verifying that guests could not receive editor permissions. A limitation is that role changes can complicate prior document access, but this is mitigated by downgrading editor access to viewer access when a user becomes a guest.

Input Validation

BearShare validates usernames, emails, passwords, filenames, and uploads before trusting them. File uploads are protected through extension whitelisting, filename sanitization, MIME validation, and file-signature checks, which is stronger than relying on extensions alone. Testing included malformed credentials, suspicious filenames, XSS-style input, path-traversal-style input, empty files, and mismatched file-content uploads. The system does not perform full malware scanning, but this is partially mitigated by layered validation and backend enforcement on both original uploads and versioned uploads.

Encryption

BearShare encrypts uploaded file contents at rest using symmetric encryption and decrypts them only when an authorized user requests access. It also supports local HTTPS testing with a self-signed certificate to verify transport protection, secure cookies, and HSTS behavior. These protections were tested by confirming that stored files were unreadable in plaintext and that HTTPS responses included the expected secure headers and cookie protections. Key management and certificate handling are simplified for local development, but this is acceptable for a course project and is partially mitigated by keeping secret files out of version control.

Session Management

BearShare uses server-side session tracking with random session tokens, inactivity expiration, and logout invalidation. Session cookies are configured with HttpOnly, SameSite=Strict, and Secure on HTTPS requests, and session creation/destruction is logged for traceability. These controls were tested by verifying logout invalidation, timeout

expiration, and cookie attributes during HTTPS use. The application does not implement advanced session fingerprinting or device binding, but this is mitigated by random tokens, secure cookies, HTTPS, and explicit session expiration.

Security Headers and Logging

BearShare strengthens browser-facing security through CSP, clickjacking protection, MIME-sniffing prevention, referrer restrictions, permissions policy, XSS-related browser controls, and HSTS on secure responses. It also records access and security logs for successful actions, denied actions, session events, lockouts, and suspicious activity, with severity levels such as INFO, WARNING, and ERROR. These controls were tested by checking response headers directly and reviewing logs after misuse attempts and normal workflows. The logging and header configuration are still development-oriented rather than production-scale, but they are mitigated by direct verification, severity-based logging, and clear documentation of deployment limitations.

5. Data Protection

BearShare treats some stored information as more sensitive than others. Credentials, session tokens, uploaded documents, role assignments, and encryption keys are the most security-sensitive data in the system. By contrast, some general metadata, such as timestamps or sanitized filenames, is less sensitive but still important for integrity and proper operation.

Additionally, because certain data is more sensitive than others, different types of data are protected in different ways. Passwords are not encrypted and later decrypted; instead, they are protected through one-way bcrypt hashing. Uploaded document contents are encrypted at rest using symmetric encryption. Metadata such as document ownership, sharing relationships, and audit records is stored in JSON files. This means the system distinguishes between data that needs reversible protection for later use and data that should only be stored as a hash.

Key management in BearShare is simplified for development. The encryption key is stored locally in a dedicated key file and used by the encryption component when protecting document contents, while TLS certificate and key files are used locally to support HTTPS testing. For a course project, this approach is acceptable as long as secret files are not committed to version control and are treated as local deployment artifacts rather than application data. In a production environment, stronger key-management practices would be required, such as environment-based secret injection, protected secret storage, or a dedicated key-management service.

BearShare currently uses soft deletion for document removal. Instead of immediately and permanently erasing all traces of a document, the system marks the document as deleted so it no longer appears in normal views and can no longer be accessed through the application. This supports auditability and reduces the risk of accidental loss during development and testing. However, secure deletion is only partially implemented, because encrypted file artifacts may still remain on disk after the document has been removed from normal application access. This limitation should be acknowledged clearly:

BearShare prevents further access through the application, but it does not yet guarantee full cryptographic or filesystem-level erasure of all stored artifacts. A production-ready design would need a stronger deletion procedure for both metadata and encrypted file contents.